```
                end_value->set_value(*last);     ⓺
            }
            std::for_each(begin,last,[addend](value_type& item)⓻
                          {
                              item+=addend;
                          });
        }
        else if(end_value)
        {
            end_value->set_value(*last);     ⓼
        }
    }
    catch(...)     ⓽
    {
        if(end_value)
        {
            end_value->set_exception(std::current_exception());⓾
        }
        else
        {
            throw;     ⓫
        }
    }
  }
};
```

In this instance, the general structure is the same as with the previous algorithms, dividing the problem into chunks, with a minimum chunk size per thread ⓬. In this case, as well as the vector of threads ⓭, you have a vector of promises ⓮, which is used to store the value of the last element in the chunk, and a vector of futures ⓯, which is used to retrieve the last value from the previous chunk. You can reserve the space for the futures ⓰ to avoid a reallocation while spawning threads, because you know how many you're going to have.

The main loop is the same as before, except this time you actually want the iterator that *points to* the last element in each block, rather than being the usual one past the end ⓱, so that you can do the forward propagation of the last element in each range. The actual processing is done in the `process_chunk` function object, which we'll look at shortly; the start and end iterators for this chunk are passed in as arguments alongside the future for the end value of the previous range (if any) and the promise to hold the end value of this range ⓲.